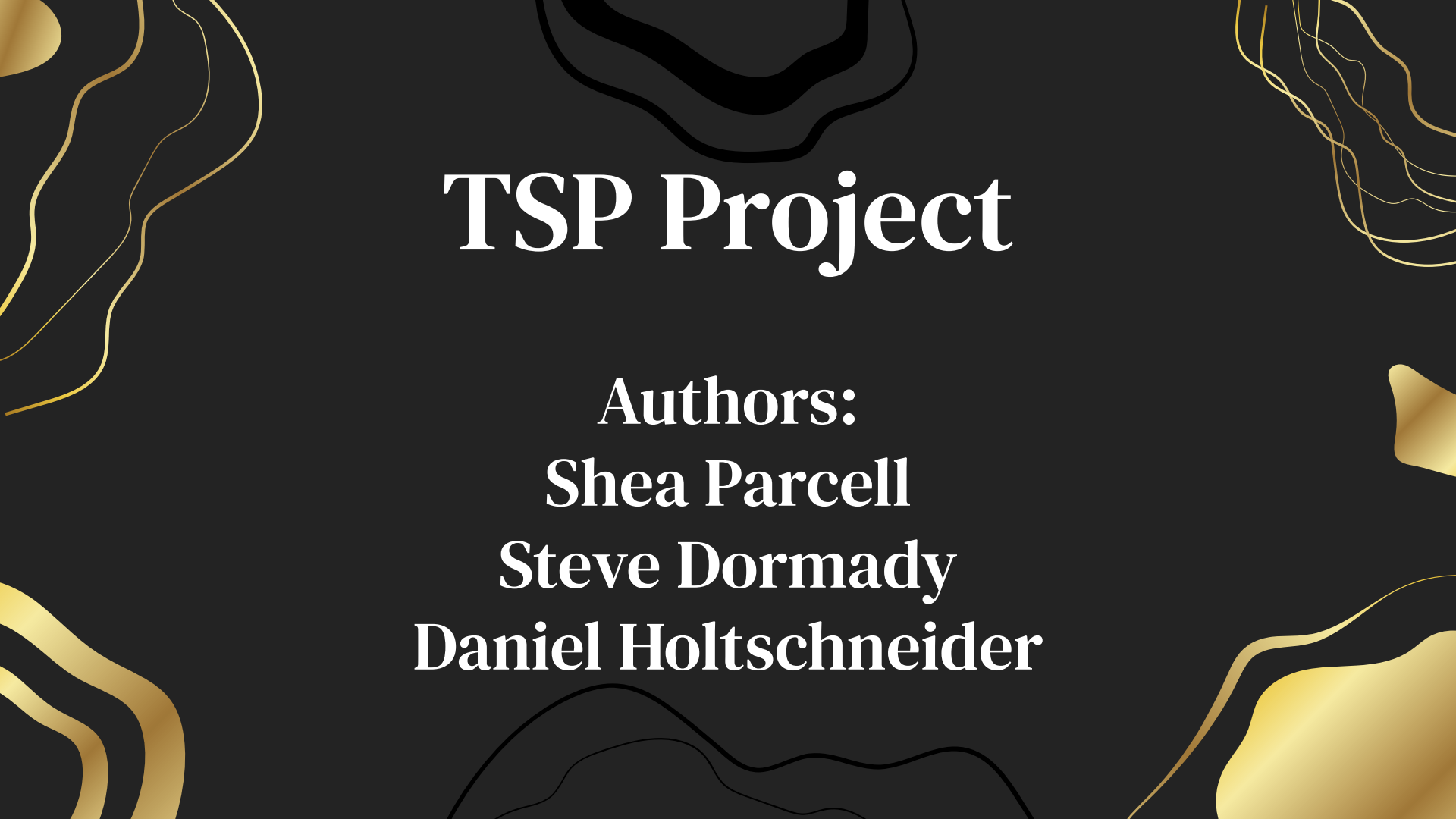




# TSP Project

Authors:

Shea Parcell

Steve Dormady

Daniel Holtschneider

# Optimization Solution

The background is a dark charcoal gray. It is decorated with several abstract, organic shapes and lines. In the corners and along the edges, there are flowing, wavy lines in a metallic gold color. Some of these lines are thin and delicate, while others are thicker and more substantial. Additionally, there are some solid black shapes, including a large, irregular blob in the top center and another in the bottom center, which appear to be part of the overall abstract design.

# Description

Traveling Salesman Problem: Visit every node in the graph once and return back to the starting node, a path/tour. Minimize and find the cost of the tour. This problem is NP-Hard meaning that as the number of nodes/cities increase the time it takes to find the optimal solution grows exponentially. No efficient solution has been found.

Optimization problem vs Decision problem:

OP: “ what’s the shortest possible cost tour that visits all cities exactly exactly once and returns to the start?”

- Output is a number of the lowest tour cost

Dec: “ is there a tour whos total length is less than or equal to  $K$ ?”

- Output is a yes/no to  $K$

# Certifier/Verifier

## Certificate (given by a prover):

- A sequence of vertices — a proposed tour.

## Question:

*“Does this certificate represent a valid tour of length  $\leq K$ ?”*

It only needs to *check* that a proposed tour is:

1. **A valid permutation** of all cities (each city visited exactly once)
2. **Returns to the start**
3. **Has total length  $\leq K$**

Each of these checks takes **polynomial time** in the number of vertices  $n = |V|$

```
CERTIFIER_TSP(G, K, tour):
```

```
    # tour is the certificate: a list of n vertices
```

```
    n = number of vertices in G
```

```
    # 1. Check the tour contains each vertex exactly once
```

```
    seen = array of n booleans initialized to false
```

```
    for v in tour:
```

```
        if seen[v] == true:
```

```
            return REJECT    # repeated city → invalid
```

```
        seen[v] = true
```

```
    # Ensure all vertices were seen
```

```
    for each vertex u in 1..n:
```

```
        if seen[u] == false:
```

```
            return REJECT    # missing city → invalid
```

```
    # 2. Compute total length of the tour, including return edge
```

```
    total = 0
```

```
    for i from 0 to n-2:
```

```
        total += weight( tour[i], tour[i+1] )
```

```
    total += weight( tour[n-1], tour[0] )
```

```
    # 3. Check if total  $\leq K$ 
```

```
    if total  $\leq K$ :
```

```
        return ACCEPT
```

```
    else:
```

```
        return REJECT
```

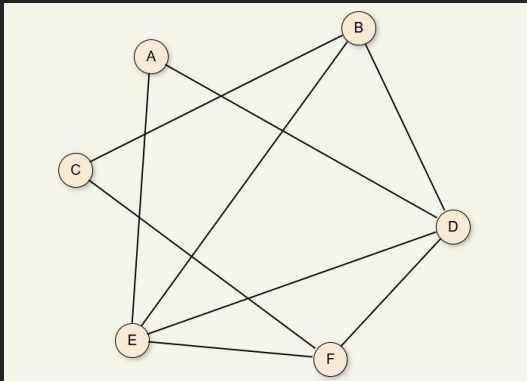
# Reduction to Hamiltonian Cycle

For all NP-Hard problems:

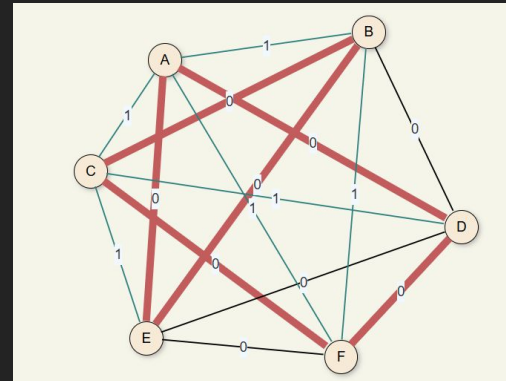
If problem Y can be reduced to problem X efficiently (in polynomial time), then  $Y \leq X$ , which means that Y is polynomial time reducible to X. X is at **least as hard** as Y.

1. Complete the graph, Make  $G'$  from Hamiltonian G, adding all the edges with weight 1, complete the edges that are not there and give them weight 0. This step can be done in polynomial time.
2. If there is a cycle that passes through all vertices exactly once, and has a length of 0, then there exists a hamiltonian cycle
3. Is there exists a hamiltonian cycle there exists a solution of the traveling salesperson problem with length = 0

$G =$



$G' =$



# Analytical Runtime

Perm will go through every  
Permutation of the vertices

Ex:

abc,bac,bca,acb,cab,cba

$n$  = # of nodes

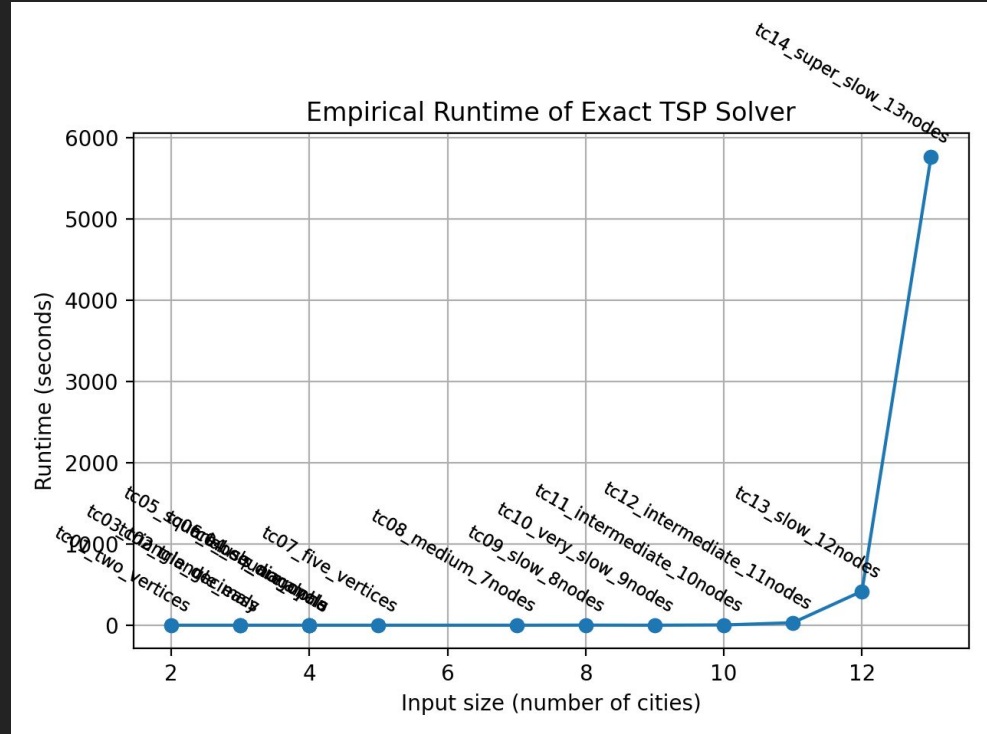
Cost of travel to node for each  
node

```
# BRUTE-FORCE TSP
# TIME COMPLEXITY:  $O(n! * n)$ 
# - itertools.permutations(vertices): generates all  $n!$  permutations
# - Inner loop (for i in range(n)): calculates cost of each  $n$ -node tour
# - Each edge lookup mat[u][v]:  $O(1)$  constant time
# DOMINANT TERM: The  $n!$  permutations dominate; for  $n=13$ , this is ~6.2 billion operations
for perm in itertools.permutations(vertices):
    cost = 0.0
    valid = True
    for i in range(n): # n edges in the tour
        u = perm[i] # all constant time operations in loop
        v = perm[(i + 1) % n] # wrap around to start
        w = mat[u][v]
        if w == INF:
            valid = False
            break
        cost += w
    if valid and cost < best_cost:
        best_cost = cost
        best_perm = perm
```

# Empirical Runtime

1200s = 20 mins

|     |                           |
|-----|---------------------------|
| 11! | = 39916800                |
| 12! | = 479001600               |
| 13! | = $6.2270208 \times 10^9$ |



# Approximate Solution

The background is a dark charcoal gray. It is decorated with several abstract, flowing, wavy lines and shapes in a metallic gold color. These elements are primarily located in the corners and along the edges, creating a sense of movement and elegance. The central text is a serif font, also in gold, which stands out against the dark background.

# Approach

- Greedily take the next node with the lowest weight that we haven't been to before (nearest neighbor)
- Give that edge a very high chance of being picked but also allow other edges to be chosen randomly
- Ensures that the result changes and we can optimize the answer
- Does a flat number of “second runs” based on input size (vertices) →

```
if V <= 200:  
    K = 10  
elif V <= 600:  
    K = 8  
elif V <= 1000:  
    K = 4  
else:  
    K = 3
```

# Pseudo Code

```
MAIN:
read V, E, edges
build distance matrix dist
choose K based on V
best_cost ← infinity
best_tour ← null
repeat K times:
    (tour, cost) ← STOCHASTIC_GREEDY(dist)
    if cost < best_cost:
        best_cost ← cost
        best_tour ← tour
convert best_tour indices to labels
print best_cost and best_tour

STOCHASTIC_GREEDY(dist):
choose random start city s
mark s visited
tour ← [s], current ← s, cost ← 0
while unvisited cities remain:
    for each unvisited v compute weight[v] = 1 / dist[current][v]
    next ← weighted-random-choice(weight)
    add next to tour and mark visited
    cost ← cost + dist[current][next]
    current ← next
cost ← cost + dist[current][s]
return (tour, cost)
```

# Runtime Analysis

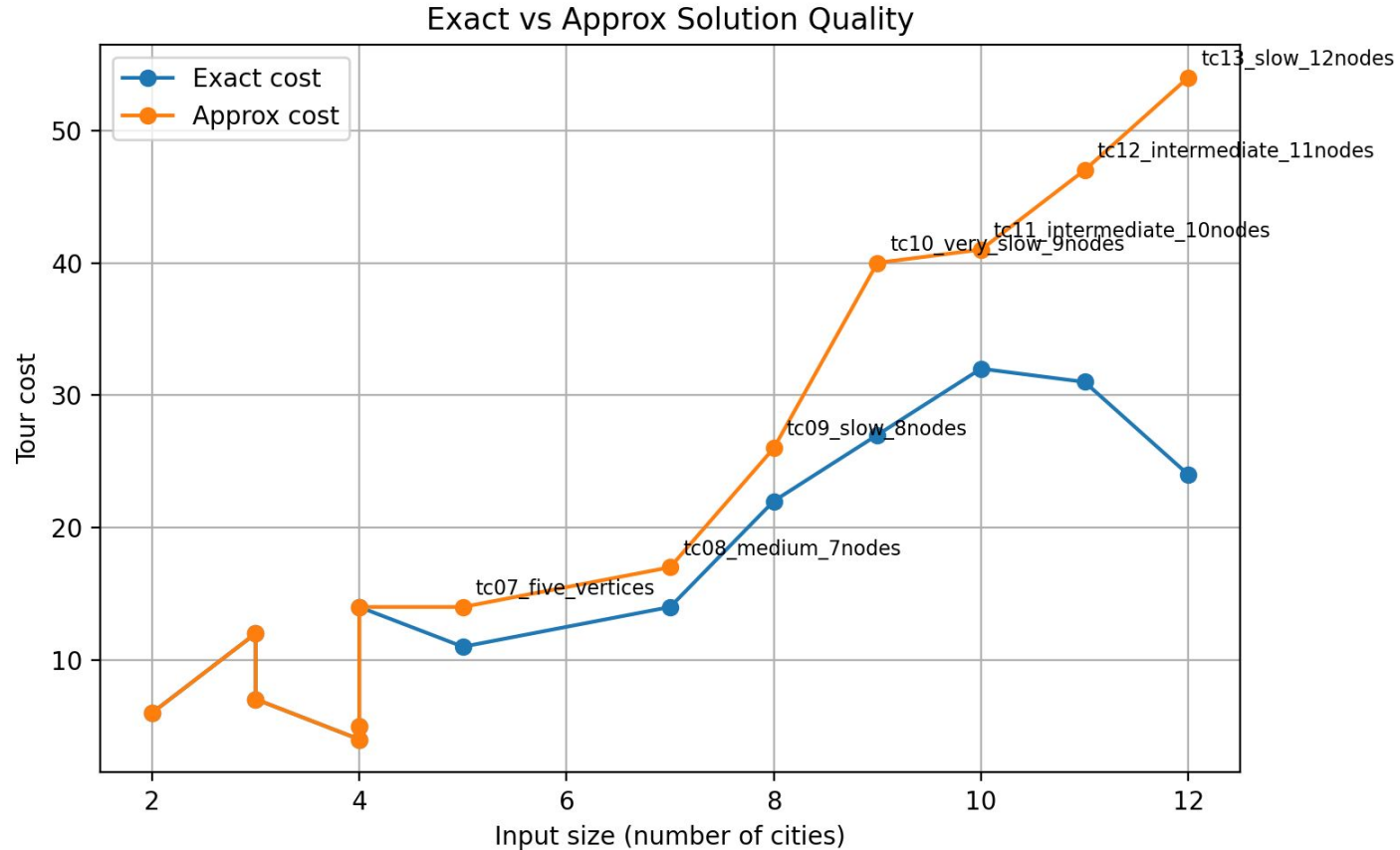
```
for _ in range(n - 1): #O(n) iterations
    candidates = []
    weights = []

    for v in range(n): #O(n)
        if not visited[v]:
            d = dist[current][v]
            w = 1.0 / (d + 1e-12)
            candidates.append(v)
            weights.append(w)

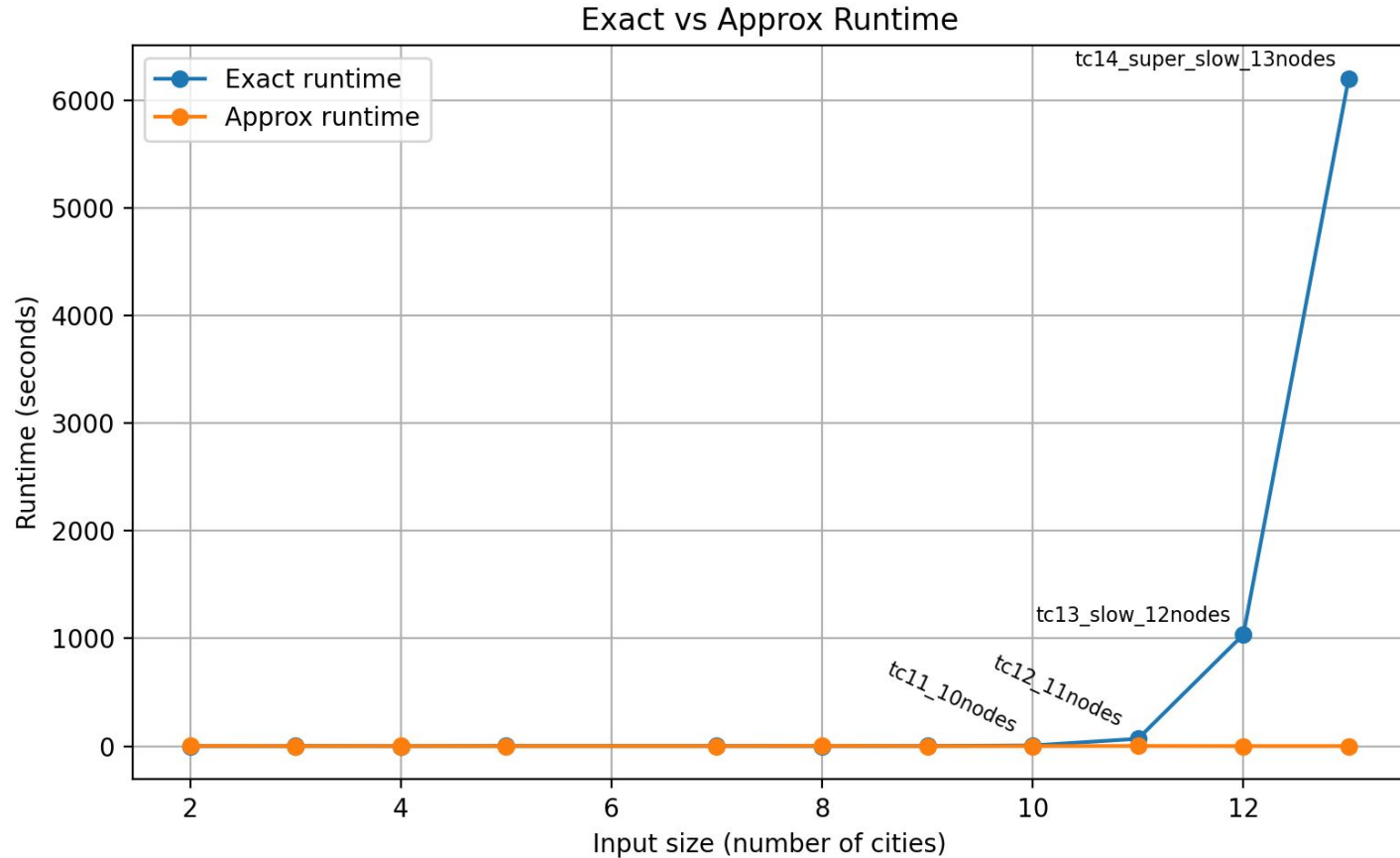
    next_city = weighted_random_choice(candidates, weights) # O(n)

    visited[next_city] = True
    tour.append(next_city)
    cost += dist[current][next_city]
    current = next_city
    # Total: O(n) + O(n) = O(n) per iteration → O(n^2) total
cost += dist[current][start]
```

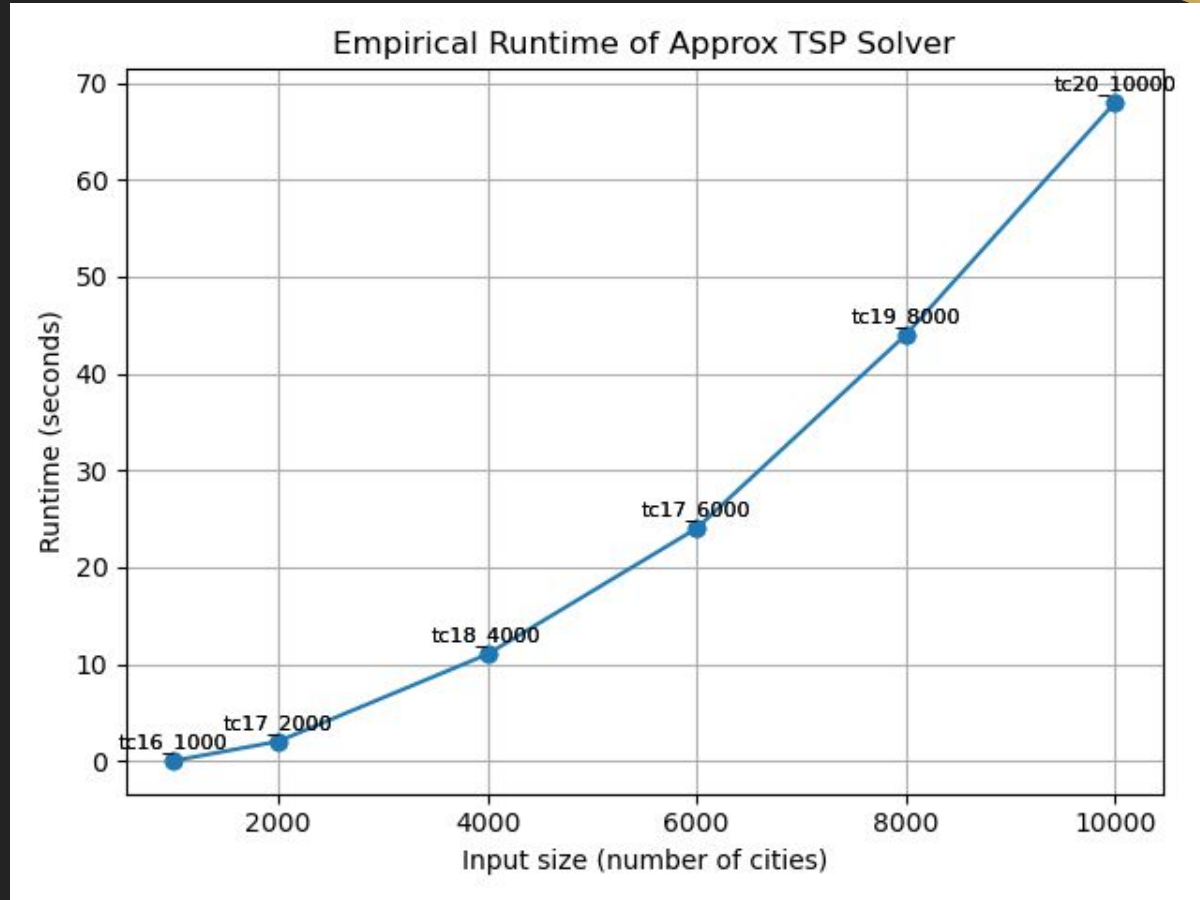
# Cost Plot (Approx vs Exact)



# Runtime Plot (Approx vs Exact)

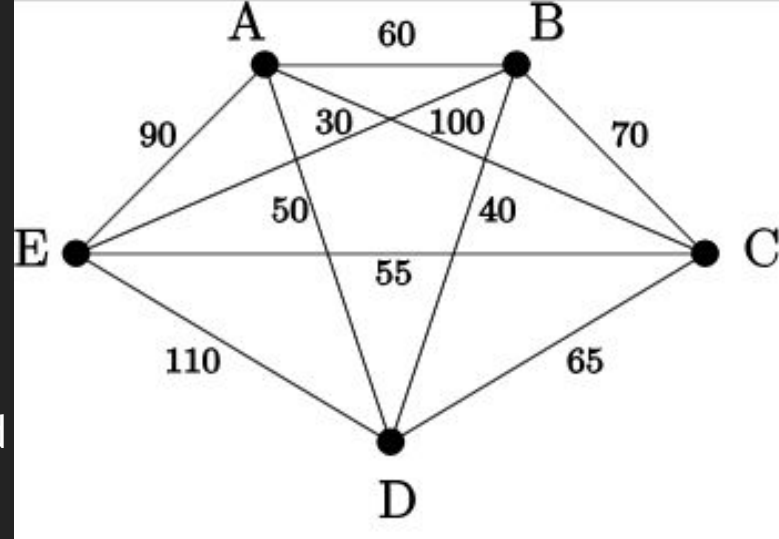


# Runtime Plot (10000 vertices)



# Lower Bound and Delta

- The lower bound of the following graph would be 237.5
- The approach used to calculate the lower bound was adding the two lowest weights of each vertex and halving it
- Approx program tour cost: 260
- The delta between the lower cost and my personal output is 22.5





# Augmented Solution

# Pseudo Code/Approach

- Like the Approximate Approach, I started with a Greedy algorithm making a complete tour, however no randomness was used.
- Then, Two-opt takes over and makes improvements. This is the key difference between the Approximate solution and this Augmented Solution.

```
# Start anywhere, repeatedly choose
the closest unvisited neighbor.
GREEDY(dist):
    start at any city
    repeatedly go to the nearest unvisited city
    return the full tour

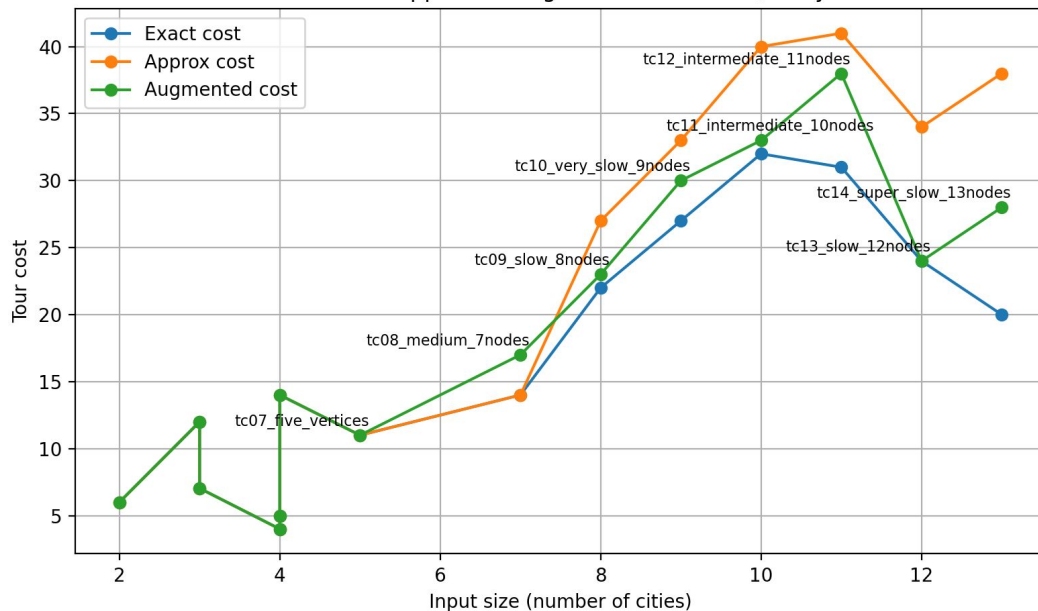
# Try reversing segments of the tour. If it
makes the tour shorter, keep it. Stop when no
improvement is found.
2OPT(tour, dist):
    repeat:
        try all edge-swap (2-opt) moves
        if any swap reduces tour length:
            apply the best-improving swap
        else:
            break
    return improved tour

# Build a fast greedy tour, then polish it with
2-opt. This gives a good approximation efficiently.
APPROX_TSP(dist):
    tour ← GREEDY(dist)
    tour ← 2OPT(tour, dist)
    return tour and its cost
```

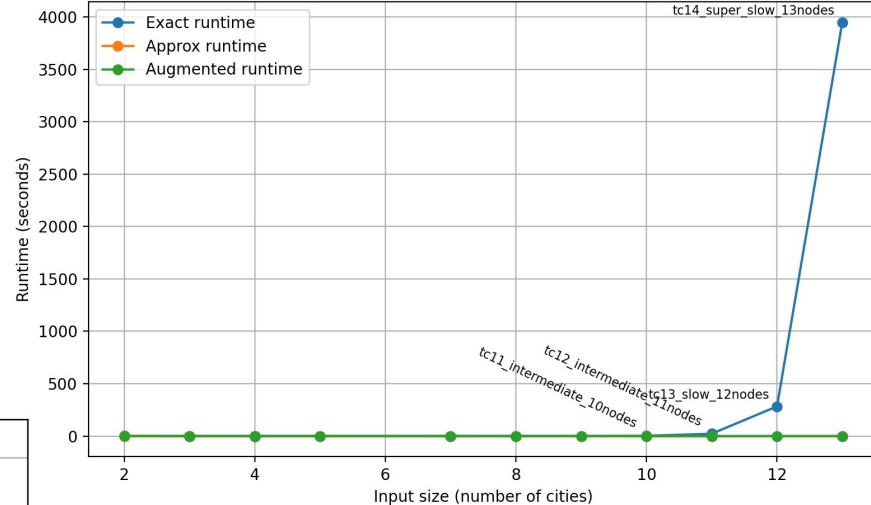
# Exact V Approx V Aug

Cost: Exact < Augmented < Approx

Exact vs Approx vs Augmented Solution Quality



Exact vs Approx vs Augmented Runtime



The graph above has a runtime of 4000 seconds, this was ran on a lab machine later in the day.

# Runtime

```
def greedy_tour(dist, start=0):
    """
    Nearest-neighbor tour starting at `start`.

    Runtime Notes:
        Let n = number of cities (len(dist)).

        - Outer loop runs (n - 1) times (we choose one new city each step) → O(n).
        - In each iteration, we scan all n cities to find the nearest unvisited neighbor → O(n).
        - Total runtime: O(n) * O(n) = O(n^2).
    """
    n = len(dist)
    visited = [False] * n # O(n) to allocate and initialize the list, doesn't effect asymptotic runtime
    tour = [start]
    visited[start] = True
    current = start

    # Outer loop: runs (n - 1) times → O(n)
    for _ in range(n - 1):
        best_v = None
        best_d = float("inf")
        row = dist[current]

        # Inner loop: scan all vertices to find nearest unvisited neighbor → O(n)
        for v in range(n):
            if not visited[v]:
                d = row[v]
                if d < best_d:
                    best_d = d
                    best_v = v
        visited[best_v] = True
        tour.append(best_v)
        current = best_v

    # Overall:
    # - Outer loop: O(n)
    # - Inner loop: O(n) per outer iteration
    # → Total: O(n^2)

    return tour
```

```
def two_opt(tour, dist, runtime_limit=None, start_time=None):
    """
    2-opt local search. Improves a tour by reversing segments.
    First-improvement strategy, stops when no improvement or time limit.
```

## Runtime Notes:

- Let n = number of cities.
- The nested (i, j) loops enumerate  $O(n^2)$  edge pairs.
- Each improving move reverses a tour segment in  $O(n)$  time.
- If R improving moves occur, total runtime is  $O(R * n^2)$ .
- Worst-case 2-opt runtime is  $O(n^3)$ , though typical performance is closer to  $O(n^2)$ .

```
if start_time is None:
```

```
    start_time = time.time()
```

```
n = len(tour)
```

```
improved = True
```

```
while improved:
```

```
    improved = False
```

```
    # Outer loop over indices i → O(n)
```

```
    for i in range(1, n - 2):
```

```
        a = tour[i - 1]
```

```
        b = tour[i]
```

```
        # Inner loop over indices j → O(n)
```

```
        for j in range(i + 1, n - 1):
```

```
            if runtime_limit is not None and time.time() - start_time > runtime_limit:
```

```
                return tour
```

```
            c = tour[j]
```

```
            d = tour[(j + 1) % n]
```

```
            # Cost change check – constant-time operations
```

```
            old_cost = dist[a][b] + dist[c][d]
```

```
            new_cost = dist[a][c] + dist[b][d]
```

```
            # If reversing segment improves the tour
```

```
            if new_cost + 1e-12 < old_cost:
```

```
                # Reversing tour[i : j+1] costs O(k), where k = j - i + 1 (worst-case O(n))
```

```
                tour[i : j + 1] = reversed(tour[i : j + 1])
```

```
                improved = True
```

```
                break # restart search from scratch (does not change asymptotic cost)
```

```
            if improved:
```

```
                break # break outer loop to restart from i = 1
```

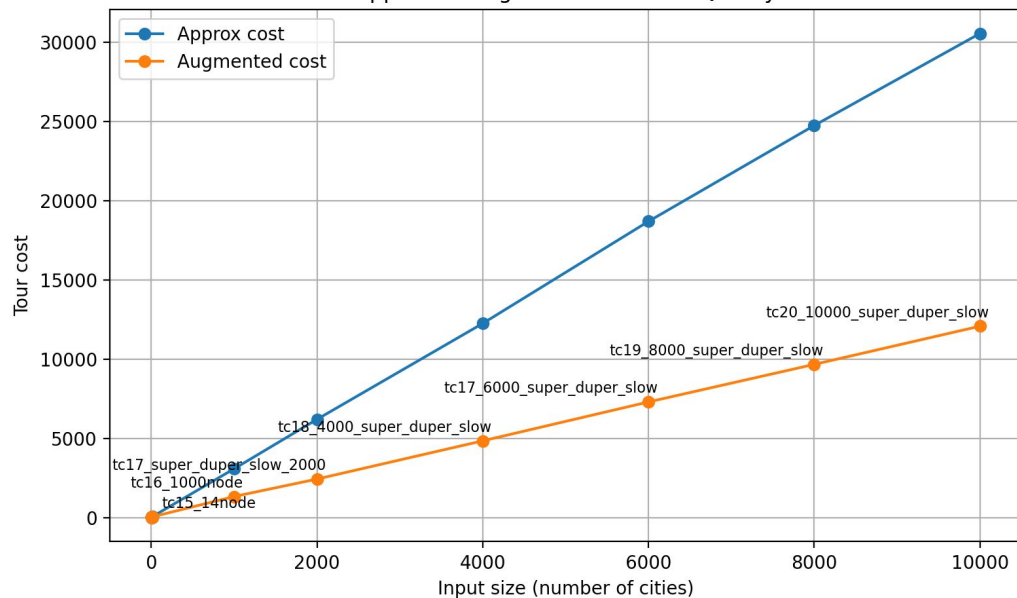
```
    return tour
```

# Approx V Aug

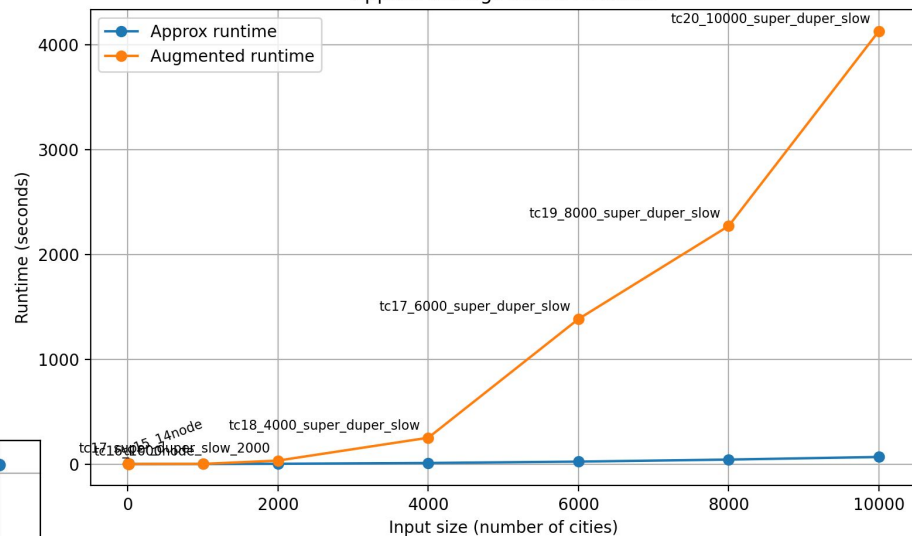
In the graph below, you can see the different slopes.

- Augmented:  $5/4$
- Approx:  $\sim 3$

Approx vs Augmented Solution Quality



Approx vs Augmented Runtime



# Optimized Pseudo Code

```
APPROX_TSP(dist, total_time):

    start_time ← now
    best_cost ← +∞
    best_tour ← None

    per_run_costs ← []      # NEW: store cost of each restart
    history       ← []      # NEW: (time, best_cost_so_far)

    while (now - start_time) < total_time:

        # NEW: randomize starting point to escape local minima
        start ← random city
        tour ← GREEDY_TOUR(dist, start)

        # Improve tour using 2-opt until no better move or time runs out
        tour ← TWO_OPT(tour, dist, time_limit = total_time)

        cost ← TOUR_COST(tour)
        per_run_costs.append(cost)

        if cost < best_cost:
            best_cost ← cost
            best_tour ← tour

        history.append( (now - start_time, best_cost) )

    return best_tour, best_cost, per_run_costs, history
```

# Optimized Runtime

```
def approx_tsp(dist, total_time=5.0):
    """
    Advanced Part E approximation:
    - Multi-restart greedy + 2-opt within a global time budget.
    - Tracks:
        * per_run_costs: cost from each restart (for variance)
        * history: (elapsed_time, best_cost_so_far) for best-vs-time plot

    Runtime analysis:
    Let n be the number of vertices.
    For each restart:
    - GREEDY_TOUR:  $O(n^2)$ 
    - TWO_OPT: worst-case  $O(n^3)$ , typically closer to  $O(n^2)$ 
    - TOUR_COST:  $O(n)$ 

    If we perform R restarts within the time budget:
    - Worst-case:  $O(R * n^3)$ 
    - In practice, R is bounded by total_time, so runtime is effectively
    min(algorithm work, time budget).
    """
    start_time = time.time()
    best_cost = float("inf")
    best_tour = None
    per_run_costs = [] # one cost per restart
    history = [] # (elapsed_time, best_cost_so_far)
    n = len(dist)

    # Multi-restart loop: number of iterations R is limited by total_time.
    while time.time() - start_time < total_time:
        # New starting tour: greedy from a random start node  $\rightarrow O(n^2)$ 
        start_node = random.randrange(n)
        tour = greedy_tour(dist, start=start_node)

        remaining = total_time - (time.time() - start_time)
        if remaining <= 0:
            break
        # Local improvement with 2-opt + worst-case  $O(n^3)$ 
        tour = two_opt(tour, dist, runtime_limit=remaining, start_time=start_time)
        # Compute cost of this tour  $\rightarrow O(n)$ 
        cost = tour_cost(tour, dist)
        per_run_costs.append(cost)

        if cost < best_cost:
            best_cost = cost
            best_tour = list(tour)

    elapsed = time.time() - start_time
    history.append((elapsed, best_cost))
```

# Aug v Optimized Aug

As you can see, the runtime has drastically improved with the optimized version, at the sacrifice of some cost efficiency.

